CP Algorithms Handbook

for the recently deceased

Stanisław Fiedler

November 11, 2026

Contents

1 Sorting and Searching 2
1.1 Quick Sort 2
1.2 Merge Sort 2
2 Dynamic Programming 3
2.1 State represenation 3
2.1.1 All subsets 3
3 Graph Algorithms 3
3.1 DFS 3
3.2 BFS 3
3.3 Shortest Path 3
3.3.1 Dijkstra 3
3.3.2 Bellman-Ford 4
3.3.3 Floyd–Warshal 4
3.4 Union Find 5
3.5 Path through all edges 5
3.5.1 Oiler cycle (path), conditions: 5
3.5.2 Implementation 6
3.6 Topo Sort 6
3.7 Flows and Cuts 7
3.7.1 Maximum Flow 7
3.7.2 Minimum Cut 8
3.7.3 Maximum Matching 9
3.8 2SAT 9
4 Range Queries 11
4.1 Segment trees 11
4.1.1 Point-Range Trees 11
4.1.2 Range-Range Trees 12
4.2 Max Subarray Sum 13
5 Tree Algorithms 14
5.1 Binary Lifitng 14
5.2 Tree Traversal 14
5.3 LCA 14
6 String Algorithms 14
6.1 Trie 14
6.2 Pattern Finding 15
6.2.1 KMP 15
6.2.2 Hashing 16
6.3 Palindormes 16
7 Mathematics 17
7.1 Small Fermat's Theorem (Modular inverse) 17
7.2 Fast Fibonacci (Matrix) 17

7.3 18
7.4 Chinese Remainder Theorem 18
8 Geometry 18
8.1 Point location 18
8.2 Point in polygon 18
8.3 Convex hull 19

1 Sorting and Searching

1.1 Quick Sort

```
void qwik_sort(int p, int q, int tab[]){
    int a = p, b = q;
    int m = tab[(p+q)/2];

    while(a <= b){
        while(m > tab[a])
            a++;
        while(m < tab[b])
            b--;
        if(a <= b){
            swap(tab[a],tab[b]);
                a++;
                b--;
        }
    }
    if(b > p)
        qwik_sort(p, b, tab);
    if(a < q)
        qwik_sort(a, q, tab);
}
```

1.2 Merge Sort

```
void merge(int left, int mid, int right, int t[]){
    int a = left, b = mid;
    int i = left;
    while(a < mid && b < right)
        tab[i++] = (t[a] < t[b] ? t[a++] : t[b++]);
    while(a < mid)
        tab[i++] = t[a++];
    while(b < right)
        tab[i++] = t[b++];
    for(int k = left; k < right; k++)
        t[k] = tab[k];
}

void mergesort(int left, int right, int t[]){
    if(left+1 != right){
        int m = (left+right)/2;
        mergesort(left, m, t);
        mergesort(m, right, t);
        merge(left, m, right, t);
    }
}
```

2 Dynamic Programming

2.1 State represenation

2.1.1 All subsets

This way you can generate all subsets in order ready for DP.

```
    int sub = whole_set;
    while(sub > 0){

        sub = (sub-1)&whole_set;
    }
```

3 Graph Algorithms

3.1 DFS

```
vector<bool> vis;

int dfs(int a) {
    vis[a] = true;

    int total = 0;
    for (int b : adj[a]) {
        if (!vis[b]) {
            total += dfs(b);
        }
    }
    return total + adj[a].size();
}
```

3.2 BFS

For BFS implementation look in Dijkstra

3.3 Shortest Path

3.3.1 Dijkstra

Dijkstra's algorithm is a greedy graph-search algorithm that finds the shortest path from a starting source node to all other nodes in a weighted graph. Does not work for negative edges.

```
priority_queue<pair<long long int ,int>> q;
dist[1] = 0;
q.push({0, 1});
while(!q.empty()){
    int a = q.top().second;
    q.pop();
    if(vis[a]) continue;
    vis[a] = true;
    for(auto v : adj[a]){
        int b = v.first;
        int w = v.second;
        if(dist[b] > dist[a] + w){
            dist[b] = dist[a] + w;
            q.push({-dist[b], b});
        }
    }
```

```
}
```

3.3.2 Bellman-Ford

Bellman–Ford algorithm is a DP graph-search algorithm that finds the shortest path from a starting source node to all other nodes in a weighted graph. Works for negative edges, and can detect negative cycle (if n-th iteration changes the distance).

```
dist[1] = 0;
for(int i = 1; i <= n-1; i++){
    for(auto e : edges){
        int a, b, w;
        tie(a, b ,w) = e;
        if(dist[b] > dist[a]+w){
            par[b] = a;
            dist[b] = dist[a]+w;
        }
    }
}
for(auto e : edges){
    int a, b, w;
    tie(a, b ,w) = e;
    if(dist[b] > dist[a]+w){
        found = true;
        node = b;
        par[b] = a;
        dist[b] = dist[a]+w;
}
```

3.3.3 Floyd–Warshal

The Floyd–Warshall algorithm is a dynamic programming method used to find the shortest paths between all pairs of vertices in a weighted directed or undirected graph.

```
long long int dist[SIZE][SIZE] {};
for(int i = 0; i <= n; i++){
    for(int k = 0; k <= n; k++){
        dist[i][k] = __INT64_MAX__/2;
    }
}
for(int i = 1; i <= n; i++){
    dist[i][i] = 0;
    for(auto v : adj[i]){
        int b = v.first;
        long long int w = v.second;
        dist[i][b] = min(w, dist[i][b]);
        dist[b][i] = min(w, dist[i][b]);
    }
}

for(int k = 1; k <= n; k++){
    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= n; j++){
            dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
        }
    }
}
```

3.4 Union Find

```cpp
int s_size[MX];
int link[MX];

int max_size = 0;
int num;

int find(int x){
    while(x != link[x])
        x = link[x];
    return x;
}

bool same(int a, int b){
    return find(a) == find(b);
}

void unite(int a, int b){
    a = find(a);
    b = find(b);
    if(s_size[a] < s_size[b])
        swap(a, b);
    s_size[a] += s_size[b];
    max_size = max(max_size, s_size[a]);
    link[b] = a;
    num--;
}

int main(){
    for(int i = 0; i < MX; i++){
        s_size[i] = 1;
        link[i] = i;
    }

    int n, m;
    cin >> n >> m;
    num = n;
    for(int i = 0; i < m; i++){
        int a, b;
        cin >> a >> b;
        if(!same(a, b))
            unite(a, b);
        cout << num << " " << max_size << endl;
    }
    return 0;
}
```

3.5 Path through all edges

3.5.1 Oiler cycle (path), conditions:

- An **undirected** graph has an Eulerian
  - **cycle** if and only if **every** vertex has **even degree**
  - **trail** if and only if exactly **zero or two** vertices have **odd degree**
- A **directed** graph has an Eulerian
  - **cycle** if and only if **every** vertex has **equal in** degree and **out** degree
  - **trail** if and only if:
    - at most one vertex has (out-degree) − (in-degree) = 1 **(one source)**

- at most one vertex has (in-degree) – (out-degree) = 1 **(one sink)**
- every other vertex has equal in-degree and out-degree

3.5.2 Implementation

Find if Eulerian trail exists (from 1 - n) in directed graph exists.

```
vector<set<int>> adj;
vector<int> in; // in-degree of node
vector<int> out; // out-degree of node

// dfs edge count

int main() {
    // read input

    // check conditions
    //   - every path is reachable dfs(1) == m
    //   - source +1 out
    //   - sink +1 in
    //   - others in == out

    vector<int> ans;

    stack<int> s;
    s.push(1);
    while (!s.empty()) {
        int a = s.top();
        if (adj[a].empty()) {
            ans.push_back(a);
            s.pop();
        } else {
            int b = *adj[a].begin();
            adj[a].erase(adj[a].begin());
            s.push(b);
        }
    }

    reverse(ans.begin(), ans.end());

    // ans has the trail
}

```

3.6 Topo Sort

```
int n, m;
stack <int> res;
vector<int> g[100007];
int vis[100007];
bool imp = false;

void dfs(int n){
    vis[n] = 1;
    for(int v : g[n]){
        if(vis[v] == 0)
            dfs(v);
        else if(vis[v] == 1)
            imp = true;
    }
    vis[n] = 2;
```

```
    res.push(n);
}

int main(){
    cin >> n >> m;
    for(int i = 0 ; i < m; i++){
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
    }

    for(int i = 1; i <= n; i++){
        if(vis[i] == 0){
            dfs(i);
        }
    }

    if(!imp){
        while(!res.empty()){
            cout << res.top() << " ";
            res.pop();
        }
    }
    else{
        cout << "IMPOSSIBLE";
    }

    return 0;
}
```

3.7 Flows and Cuts

All problems in this section can be solved using the same basic algorithm defined in Section 3.7.1.

3.7.1 Maximum Flow

This implementation of Ford-Fulkerson algorithm (known as Edmonds-Karp algorithm) uses BFS to check if it's possible to expand the flow through the graph and assign depths from the source. Later it uses DFS to expand te flow.

```
ll flow[MX][MX];
ll used[MX][MX];

vector<int> adj[MX];
vector<int> radj[MX];

bool vis[MX];

bool bfs(int s, int lvl[], const int n){
    queue<int> q;
    q.push(s);
    lvl[s] = 1;
    while(!q.empty()){
        int p = q.front();
        q.pop();
        for(int v : adj[p]){
            if(lvl[v] == 0 and flow[p][v] - used[p][v] > 0){
                lvl[v] = lvl[p]+1;
                q.push(v);
            }
```

```
        }
        for(int v : radj[p]){
            if(lvl[v] == 0 and used[v][p] > 0){
                lvl[v] = lvl[p]+1;
                q.push(v);
            }
        }
    }
    return lvl[n] > 0;
}

int dfs(int x, ll val, const int lvl[], const int n){
    vis[x] = true;
    if(x == n){
        return val;
    }
    for(int v : adj[x]){
        if(lvl[v] == lvl[x]+1 and flow[x][v] - used[x][v] > 0 and !vis[v]){
            int r = dfs(v, min(val, flow[x][v]-used[x][v]), lvl, n);
            if(r > 0){
                used[x][v] += r;
                return r;
            }
        }
    }
    for(int v : radj[x]){
        if(lvl[v] == lvl[x]+1 and used[v][x] > 0 and !vis[v]){
            int r = dfs(v, min(val, used[v][x]), lvl, n);
            if(r > 0){
                used[v][x] -= r;
                return r;
            }
        }
    }
    return 0;
}
```

To find the flow:

```
    long long int res = 0;

    while(bfs(1, lvl, n)){
        res += dfs(1, __INT_MAX__, lvl, n);
        for(int i = 0 ; i < MX; i++){
            vis[i] = false;
            lvl[i] = 0;
        }
    }
```

3.7.2 Minimum Cut

To find Minimum Cut in a graph we need to find the Maximum Flow and check which edges connect two created disjont sets of nodes. This modification to finds them:

```
    long long int res = 0;

    while(bfs(1, lvl, n)){
        res += dfs(1, __INT_MAX__, lvl, n);
```

```
        for(int i = 0 ; i < MX; i++){
            vis[i] = false;
            lvl[i] = 0;
        }
    }
    cout << res << endl;
    for(int i = 0 ; i < MX; i++){
        vis[i] = false;
        lvl[i] = 0;
    }

    bfs(1, lvl, n);
    for(int a = 1; a <= n; a++){
        if(lvl[a] > 0){
            for(int b : adj[a]){
                if(lvl[b] == 0){
                    cout << a << " " << b << endl;
                }
            }
        }
    }
```

3.7.3 Maximum Matching

3.8 2SAT

Given logical formula in the conjunctive normal form:

$$(a_1 \vee b_1) \wedge (a_2 \vee b_2) \wedge \cdots \wedge (a_n \vee b_n)$$

we can eliminate disjunction by repacing each $(a_i \vee b_i)$ element with pair:

$$\neg a_i \rightarrow b_i \wedge \neg b_i \rightarrow a_i$$

```
vector<int> adj[MX];
vector<int> Tadj[MX];
stack<int> scc_stack;

int scc[MX];

bool vis[MX];
int scc_val[MX];

int not_node(int a){
    if(!(a&1))
        return a+1;
    else
        return a-1;
}

void dfs1(int a){
    vis[a] = true;
    for(int b : adj[a]){
        if(!vis[b])
            dfs1(b);
    }
    scc_stack.push(a);
}

```

```
26 void dfs2(int a, int scc_n){
27     vis[a] = true;
28     for(int b : Tadj[a]){
29         if(!vis[b])
30             dfs2(b, scc_n);
31     }
32     scc[a] = scc_n;
33 }
34 
35 void zero(){
36     for(int i = 0 ; i < MX; i++)
37         vis[i] = false;
38 }
39 
40 int main(){
41     int n, m;
42     cin >> n >> m;
43     for(int i = 0 ; i < n; i++){
44         char op;
45         int a, b;
46         cin >> op >> a;
47         if(op == '+'){
48             a *= 2;
49         }
50         else{
51             a *= 2;
52             a++;
53         }
54         cin >> op >> b;
55         if(op == '+'){
56             b *= 2;
57         }
58         else{
59             b *= 2;
60             b++;
61         }
62         Tadj[a].push_back(not_node(b));
63         Tadj[b].push_back(not_node(a));
64         adj[not_node(b)].push_back(a);
65         adj[not_node(a)].push_back(b);
66     }
67 
68     for(int i = 1 ; i <= 2*m+1; i++){
69         if(!vis[i])
70             dfs1(i);
71     }
72 
73     zero();
74     int scc_n = 1;
75     while(!scc_stack.empty()){
76         int a = scc_stack.top();
77         scc_stack.pop();
78         if(scc[a] == 0){
79             dfs2(a, scc_n);
80             scc_n++;
81         }
82     }
83 
84     for(int i = 2; i <= m*2; i+=2){
85         if(scc[i] == scc[i+1]){
```

```
            cout << "IMPOSSIBLE" << endl;
            return 0;
        }
    }
    // 1 - false; 2 - true;
    for(int i = 2; i <= 2*m; i+=2){
        int a = scc[i];
        int b = scc[i+1];
        if(scc_val[a] != 0){
            scc_val[b] = (scc_val[a] == 1 ? 2 : 1);
        }
        if(scc_val[b] != 0){
            scc_val[a] = (scc_val[b] == 1 ? 2 : 1);
        }
        if(a < b){
            scc_val[a] = 1;
            scc_val[b] = 2;
        }
        else{
            scc_val[a] = 2;
            scc_val[b] = 1;
        }
    }

    for(int i = 2; i <= m*2; i+=2)
        cout << (scc_val[scc[i]] == 2 ? "+" : "-") << " ";

    return 0;
}
```

4 Range Queries

4.1 Segment trees

BASE size table:

a	$2 \cdot 10^5$	$5 \cdot 10^5$	10^6
$\log_2 a$	18	19	20

4.1.1 Point-Range Trees

```
int sal[2*BASE];

void add(int p, int val){
    p += BASE;
    sal[p] += val;
    while(p > 0){
        p >>= 1;
        sal[p] = sal[2*p] + sal[2*p+1];
    }
}

int query(int p, int q){
    int ret =0 ;
    p += BASE -1;
    q += BASE +1;
    while(p>>1 != q>>1){
        if(not(p&1)){
```

```
            ret += sal[p+1];
        }
        if(q&1){
            ret += sal[q-1];
        }
        p >>= 1;
        q >>= 1;
    }
    return ret;
}
```

4.1.2 Range-Range Trees

1. range ADD insert, range MAX value query

```
void max_add(int a, int b, int val, int k=1, int x=0, int y=BASE-1){
    if(a <= x and y <= b){
        max_tree[k][1] += val;
        return;
    }
    max_tree[2*k][1] += max_tree[k][1];
    max_tree[2*k+1][1] += max_tree[k][1];
    max_tree[k][0] += max_tree[k][1];
    max_tree[k][1] = 0;

    int d = (x+y)/2;
    if(a <= d){
        max_add(a, b, val, 2*k, x, d);
        }
    if(b > d){
        max_add(a, b, val, 2*k+1, d+1, y);
    }
    max_tree[k][0] = max(max_tree[2*k+1][0] + max_tree[2*k+1][1],
max_tree[2*k][0] + max_tree[2*k][1]);

}
```

```
ll max_query(int a, int b, int k=1, int x=0, int y=BASE-1){
    if(a <= x and y <= b){
        return max_tree[k][0] + max_tree[k][1];
    }
    max_tree[2*k][1] += max_tree[k][1];
    max_tree[2*k+1][1] += max_tree[k][1];
    max_tree[k][0] += max_tree[k][1];
    max_tree[k][1] = 0;
    int d = (x+y)/2;
    ll ret = -INF;
    if(a <= d){
        ret = max(ret, max_query(a, b, 2*k, x, d));
    }
    if(b > d){
        ret = max(ret, max_query(a, b, 2*k+1, d+1, y));
    }
    return ret;
}
```

2. range ADD insert, range SUM qyerry

```
void sum_add(int a, int b, int val, int k=1, int x=0, int y=BASE-1){
    if(a <= x and y <= b){
        sum_tree[k][1] += val;
        return;
    }
    sum_tree[2*k][1] += sum_tree[k][1];
    sum_tree[2*k+1][1] += sum_tree[k][1];
    sum_tree[k][0] += sum_tree[k][1]*(y-x+1);
    sum_tree[k][1] = 0;
    int d = (x+y)/2;
    if(a <= d){
        int w = (d <= b ? d : b)+1;
        w -= (x <= a ? a : x);
        sum_tree[k][0] += val*(w);
        sum_add(a, b, val, 2*k, x, d);
    }
    if(b > d){
        int w = (y <= b ? y : b)+1;
        w -= (d+1 <= a ? a : d+1);
        sum_tree[k][0] += val*(w);
        sum_add(a, b, val, 2*k+1, d+1, y);
    }
}
```

```
ll sum_query(int a, int b, int k=1, int x=0, int y=BASE-1){
    if(a <= x and y <= b){
        return sum_tree[k][0] + sum_tree[k][1]*(y-x+1);
    }
    sum_tree[2*k][1] += sum_tree[k][1];
    sum_tree[2*k+1][1] += sum_tree[k][1];
    sum_tree[k][0] += sum_tree[k][1]*(y-x+1);
    sum_tree[k][1] = 0;
    int d = (x+y)/2;
    ll ret = 0;
    if(a <= d){
        ret += sum_query(a, b, 2*k, x, d);
    }
    if(b > d){
        ret += sum_query(a, b, 2*k+1, d+1, y);
    }
    return ret;
}
int sal[2*BASE];
```

4.2 Max Subarray Sum

```
ll ssum[2*BASE];
ll prfx[2*BASE];
ll sufx[2*BASE];
ll tsum[2*BASE];

void insert(int val, int p){
    p+=BASE;
    ssum[p] = val;
    prfx[p] = val;
    sufx[p] = val;
    tsum[p] = val;
    int l, r;
```

```
    while(p){
        p >>= 1;
        l = 2*p;
        r = 2*p+1;
        tsum[p] = tsum[l] + tsum[r];
        prfx[p] = max(prfx[l], tsum[l] + prfx[r]);
        sufx[p] = max(sufx[l] + tsum[r], sufx[r]);
        ssum[p] = max(sufx[l] + prfx[r], max(ssum[l], ssum[r]));
    }
}
```

5 Tree Algorithms

5.1 Binary Lifitng

```
int main(){
    int n, q;
    cin >> n >> q;
    boss[1][0] = 0;
    for(int i = 2; i <= n; i++){
        cin >> boss[i][0];
    }
    for(int i = 1; i < LOG_MX; i++){
        for(int emp = 1; emp <= MX; emp++){
            boss[emp][i] = boss[boss[emp][i-1]][i-1];
        }
    }
```

5.2 Tree Traversal

```
vector <int> adj[MX];
int subtree[MX];
int flat[MX];
int vals[MX];

int idx = 1;
int dfs(int a, int p){
    flat[a] = idx;
    idx++;
    for(int b : adj[a]){
        if(b == p)
            continue;
        subtree[a] += dfs(b, a)+1;
    }
    return subtree[a];
}
```

5.3 LCA

LCA can be found using binary lifting or by building a segment tree build on pre- and postorder.

6 String Algorithms

6.1 Trie

```
int trie[TMX][30];
bool stop[TMX];
```

```cpp
int next_node = 1;
int dp[MX];

void insert(string s){
    int idx = 0;
    for(char c : s){
        if(trie[idx][c-'a'] == 0){
            trie[idx][c-'a'] = next_node;
            next_node ++;
        }
        idx = trie[idx][c-'a'];
    }
    stop[idx] = true;
}

int main(){
    string text;
    int n;
    cin >> text >> n;
    for(int i = 0; i < n; i++){
        string s;
        cin >> s;
        insert(s);
    }
```

6.2 Pattern Finding

6.2.1 KMP

The Knuth-Morris-Pratt (KMP) algorithm is a linear-time string-searching algorithm that finds occurrences of a word W (pattern) inside a main text string S

```cpp
int pi[MX];

int main(){
    string text, pattern;
    cin >> text >> pattern;
    string s = pattern + "+" + text;
    for(int i = 1; i < s.size(); i++){
        int j = pi[i-1];
        while(j > 0 and s[i] != s[j])
            j = pi[j-1];
        if(s[i] == s[j])
            j++;
        pi[i] = j;
    }

    int res = 0;
    for(int i = 0; i < s.size(); i++){
        if(pi[i] == pattern.size())
            res++;
    }

    cout << res;
    return 0;
```

6.2.2 Hashing

```cpp
const int MOD = 1e9+9;
const int P = 9973;
const int MX = 1e6+7;

ll ppow[MX];
ll pfx_hash[MX];

int res = 0;

int main(){
    string s, pattern;
    cin >> s >> pattern;
    ppow[0] = 1;
    for(int i = 1 ; i < MX; i++){
       ppow[i] = (ppow[i-1]*P)%MOD;
    }

    ll pattern_hash = 0;
    for(int i = 0; i < pattern.size(); i++){
        pattern_hash = (pattern_hash + (pattern[i]-'a'+1)*ppow[i])%MOD;
    }

    for(int i = 0; i < s.size(); i++){
        pfx_hash[i+1] = (pfx_hash[i] + (s[i]-'a'+1)*ppow[i])%MOD;
    }

    for(int i = 0; i+pattern.size()-1 < s.size(); i++){
        int a = i, b = i+pattern.size();
        ll sub_hash = pfx_hash[b] - pfx_hash[a]+MOD;
        sub_hash %= MOD;
        if(sub_hash == (pattern_hash*ppow[a])%MOD){
            res++;
        }
    }
    cout << res << endl;
    return 0;
```

6.3 Palindormes

Fiding longest palindromic substring.

```cpp
int p[MX];

int main(){
    string txt1, txt = "#";
    cin >> txt1;
    for(char c : txt1){
        string s{c};
        txt += s + "#";
    }
    txt = "^" + txt + "$";
    int l = 1, r = 1;
    for(int i = 1; i < txt.size(); i++){
        p[i] = max(0, min(r-i, p[l+r-i]));
        while(txt[i-p[i]] == txt[i+p[i]])
            p[i]++;
        if(i+p[i] > r){
            l = i-p[i];
```

```
            r = i+p[i];
        }
    }
    int maxi = 0;
    int imaxi;
    for(int i = 0; i < txt.size(); i++){
        if(p[i] > maxi){
            maxi = p[i];
            imaxi = i;
        }
    }
    string res = txt.substr(imaxi-maxi+1,maxi*2-1);
    for(auto c : res){
        if(c != '#')
            cout << c;
    }
```

7 Mathematics

7.1 Small Fermat's Theorem (Modular inverse)

If m is prime, then: $x^{-1} \bmod m = x^{m-2} \bmod m$

$$x^{\varphi(m)} \bmod m = x^{k \bmod (m-1)} \bmod m$$

```
int exp(int a, int b, int MOD){
    if(b == 0)
        return 1;
    if(b & 1){
        return ((ll)a * exp(a, b-1, MOD))%MOD;
    }
    ll tmp = exp(a, b/2, MOD);
    return (tmp*tmp)%MOD;
}
```

7.2 Fast Fibonacci (Matrix)

```
void mul(ll a[][2], ll b[][2]){
    ll res[2][2];
    res[0][0] = a[0][0]*b[0][0] + a[0][1]*b[1][0];
    res[0][1] = a[0][0]*b[0][1] + a[0][1]*b[1][1];
    res[1][0] = a[1][0]*b[0][0] + a[1][1]*b[1][0];
    res[1][1] = a[1][0]*b[0][1] + a[1][1]*b[1][1];

    a[0][0] = res[0][0] % MOD;
    a[0][1] = res[0][1] % MOD;
    a[1][0] = res[1][0] % MOD;
    a[1][1] = res[1][1] % MOD;
}

int main(){
    ll n;
    cin >> n;
    ll m_pow[2][2] =
            {{0, 1},
             {1, 1}};
    ll m[2][2] =
```

```
            {{1, 0},
             {0, 1}};

    auto pow = bitset<64> (n);

    for(int i = 0 ; i < 64; i++){
        if(pow[i]){
            mul(m,m_pow);
        }
        mul(m_pow, m_pow);
    }

    cout << m[0][1];

    return 0;
}
```

7.3

7.4 Chinese Remainder Theorem

8 Geometry

8.1 Point location

```
ll cros = cross(a, b, p);
if(cros > 0)
    cout << "LEFT\n";
else if(cros == 0)
    cout << "TOUCH\n";
else
    cout << "RIGHT\n";
```

8.2 Point in polygon

```
int inter(pii a, pii b, pii c, pii d){
    pii p1 = {min(a.first, b.first), max(a.second, b.second)};
    pii p2 = {max(a.first, b.first), min(a.second, b.second)};
    pii p3 = {min(c.first, d.first), max(c.second, d.second)};
    pii p4 = {max(c.first, d.first), min(c.second, d.second)};

    if (p3.first > p2.first || p4.first < p1.first || p3.second < p2.second ||
p4.second > p1.second)
        return 0;

    if (sign(cross(a,b,c)) * sign(cross(a,b,d)) <= 0 && sign(cross(c,d,a)) *
sign(cross(c,d,b)) <= 0) {
        return 1;
    }

    return 0;
}

bool point_on_line(pii a, pii b, pii p){
    int c = cross(a, b ,p);
    pii ld = {min(a.first, b.first), min(a.second, b.second)};
    pii pg = {max(a.first, b.first), max(a.second, b.second)};
    if(c == 0){
```

```
        if(!(p.first < ld.first || p.second < ld.second || p.first > pg.first
|| p.second > pg.second))
            return true;
    }
    return false;
}

int count = 0;
bool boundary = false;
for(int i = 0; i < n; i++){
    if (point_on_line(polygon[i], polygon[(i == n-1 ? 0 : i+1)], p)){
        boundary = true;
    }
    if (point_on_line(p, out, polygon[i])){
        if(cross(p, out, polygon[(i == 0 ? n-1 : i-1)]) * cross(p,
out,polygon[(i == n-1 ? 0 : i+1)]) < 0){
            count --;
        }
    }
    count += inter(p, out, polygon[i], polygon[(i == n-1 ? 0 : i+1)]);
}

if (boundary){
    cout << "BOUNDARY" << endl;
}
else if (count & 1){
    cout << "INSIDE" << endl;
}
else {
    cout << "OUTSIDE" << endl;
}
```

8.3 Convex hull

Only finds the smallest convex hull. Points on the hull are omitted.

```
ll cross2(pii a, pii b) {
    return (ll)a.first * b.second - (ll)a.second * b.first;
}

int cross3(pii a, pii b, pii c){
    pii va, vb;
    va.first = b.first-a.first;
    va.second = b.second-a.second;
    vb.first = c.first-b.first;
    vb.second = c.second-b.second;
    return sign((ll)va.first*vb.second - (ll)vb.first*va.second);
}

sort(points.begin(), points.end(), [](pii l, pii r){
    int o = cross3(l, r, p0);
    if (o == 0) {
        return norm(sub(p0,l)) < norm(sub(p0,r));
    }
    return o < 0;
});

vector<pii> hull;

for (int i = 0; i < n; i++){
```

```
    while(hull.size() > 1 && cross3(hull[hull.size()-2], hull[hull.size()-1],
points[i]) >= 0){
        hull.pop_back();
    }
    hull.push_back(points[i]);
}
```